

PlasmaFold: An Efficient and Scalable Layer 2 with Client-Side Proving

Pierre Daix-Moreux¹ and Chengru Zhang^{1,2}

¹Ethereum Foundation

²The University of Hong Kong

pierre@ethereum.org, u3008875@connect.hku.hk

Abstract

Despite the growing popularity of blockchains, their scalability remains a significant challenge. Layer-2s (L2s) aim to address this by introducing an operator to process transactions off-chain and post compact summaries to the Layer-1 (L1). However, existing L2 designs struggle with unsatisfactory throughput improvements, complex exit games, limited data availability, stringent synchronization requirements or high computational overhead for users.

This paper introduces **PlasmaFold**, a novel L2 designed to overcome each of those limitations. **PlasmaFold** utilizes a hybrid architecture: an operator (aggregator) generates proofs on server side for the honest construction of blocks, while users maintain balance proofs on their own devices. This separation of concerns enables instant, non-interactive exits via balance proofs, while block proofs handle most of the validations, minimizing users' costs. By leveraging Incrementally Verifiable Computation (IVC), **PlasmaFold** achieves both instant synchronization and concrete efficiency. Using **PlasmaFold** proving routines, users can update their balance proofs within a browser in under 1 second per transaction using less than 1 GB of RAM. Our construction does not impact scaling, we show that it remains possible to leverage recent verifiable plasma research designed to keep a minimal on-chain footprint, thereby enabling **PlasmaFold** to reach a theoretical throughput of over 14000 transactions per second.

1 Introduction

Blockchains have gained significant attention in recent years, offering a decentralized and secure way to store and transfer digital assets. However, their decentralized nature often leads to challenges in terms of scalability, which results in long confirmation times and high transaction fees, hindering their widespread adoption.

An important metric for evaluating the scalability is *throughput*, which is measured as the number of transactions per second (TPS) that a system can handle. Traditional payment systems usually feature very high TPS (e.g., Visa's claims a > 65000 TPS capacity), which can be further scaled horizontally by augmenting server capacity. In contrast, public blockchains such as Ethereum

handle around 20 TPS. Although their throughput is determined by different factors, as shown in Equation (1), one fundamental limit remains the blockchain’s block size rather than its nodes’ computation power. Indeed, nodes’ computation power in a blockchain can be increased by utilizing better hardware. On the other hand, consensus protocols typically enforce conservative block size limits, as larger blocks usually increase propagation time and fork risks.

$$\text{Throughput} = \frac{\text{Block Size}}{\text{Transaction Size} \times \text{Block Time}} \quad (1)$$

Various solutions have been proposed to increase the throughput of blockchains. One has been L2 chains, which consist in offloading to an untrusted prover the task of block building. Since then, various designs with different security, data availability and user computation overheads have been put forth [28]. For instance, an *optimistic* L2 consists in a prover batching transactions and posting blocks onto the base L1 chain, along with transaction data, allowing any user to re-execute transactions and potentially engage in an interactive fraud-proof challenge with the offchain block builder [17, 24, 31]. Similarly, others leveraged succinct non-interactive arguments of knowledge (SNARKs) to attest to the correctness of executing a transaction batch [4, 10, 19, 22] and creating L2 chains colloquially known as “zkrollups” or “validity rollups”. However, validity and optimistic rollups are required to post each transaction’s data before landing it on the L1 [25, 34], ensuring users’ ability to rebuild and exit the current rollup state if needed [29, 33]. Although compressed, posting transactions’ data limits the L2 throughput [28]. While validiums and plasma setups cut down the required amount of onchain posted data, they also limit L2’s users from reconstructing the chain state [14, 15, 26]. In that line of work, Intmax2 [27] recently sought to combine plasma with SNARKs to obtain both high throughput and instant settlement from validity proofs. This new stream of work introduced new primitives making it possible to reach two important goals: a high theoretical TPS while, in contrast with previous plasma setups, allowing users to be offline. However, Intmax2 requires users to run costly proof-carrying data proving routines and synchronization proofs for each block, degrading the overall user experience.

The limitations of existing solutions raise the following question: *Can we design a scalability solution for blockchains that achieves high throughput while supporting fast exits, no online requirements, instant synchronization to the chain tip while maintaining low user computation overhead?*

1.1 Contributions

In this paper, we give a positive answer to this question by introducing **PlasmaFold**, a new L2 solution for Ethereum. The concrete contributions of this paper are as follows:

PlasmaFold: A Scalable Ethereum L2 with instant synchronization and efficient proving routines. Our contributions are twofold: (1) we introduce folding schemes based proving routines to obtain efficient proving speed and RAM usage results with respect to our L2 design and (2) obtain as a byproduct of our construction an instant synchronization property, improving over Intmax2’s proving costs and requirement for running synchronization proofs.

Hybrid Architecture. At the core of PlasmaFold lies a novel architecture that combines *client-side proving* and *server-side proving*. This design distributes the computational burden between the aggregator and users: the aggregator only needs to maintain a *block proof* (to ensure correctness when constructing blocks), without tracking each L2 user’s balance; users, on the other hand, are responsible for updating their own *balance proofs* (to guarantee the correctness of their balance) but do not need to demonstrate the full transaction history.

To reduce the RAM costs for both the aggregator and users, PlasmaFold employs IVC based on *folding schemes*, which enables the aggregator and users to update their proofs efficiently.

Finally, in addition to the hybrid mode, one can also deploy PlasmaFold in validium mode, where the user no longer needs to maintain a local balance proof but instead relies on the aggregator to exit the L2.

Instant synchronization. Although Intmax2 users are not required to be online, it is still required for them to run a synchronization proof routine, making their balance updates cumbersome. In PlasmaFold, we provide the same offline guarantee while providing an instant synchronization property. To obtain the ability for users to remain offline, we leverage Intmax2’s data availability procedure which consists in the aggregator collecting signatures from users, each attesting to having received necessary offchain data for proving their transaction inclusion [27]. However, to allow users to completely skip irrelevant blocks, we design PlasmaFold using IVC. Now, users only need to prove updates for blocks containing transactions they sent or receive, they do not need to run any synchronization proofs.

Implementation and Evaluation. We provide a prototype implementation of PlasmaFold with an efficient hybrid stack for client-side and server-side proving. We then evaluate the performance of PlasmaFold. Our results demonstrate that PlasmaFold features highly efficient proving routines. To update a balance proof with 5 transactions, a user only needs to run a WebAssembly program on their web browser that takes 5.3 seconds and 579 MB of memory when running with a circuit friendly hash function. For each transaction, the aggregator can also update the block proof with less than 1.5 seconds and 5 GB of memory.

Theoretical throughput. We also show that our instant synchronization property and efficient proving routines designs do not impact PlasmaFold TPS. By leveraging Intmax2’s short-id trick [27], PlasmaFold corresponding data on the L1 only takes up ~ 4 bytes, leading to an equivalent theoretical throughput of over 14000 TPS under the restriction of a 784 KB block size.

1.2 Overview of PlasmaFold

We now provide a high-level overview of PlasmaFold, which specifies the interaction among three entities: *users*, the *aggregator*, and the *rollup contract* deployed on the L1 blockchain. In PlasmaFold, users can make *transactions* using the Unspent Transaction Output (UTXO) model to transfer funds to others, while the aggregator is responsible for collecting these transactions and producing

blocks. The rollup contract verifies the validity of these blocks and updates its state accordingly.

To understand how PlasmaFold is constructed, we first introduce the intuition behind our design through a series of progressively refined constructions.

Strawman 1. As our first strawman construction, we consider the simplest case in which the contract directly manages a list of UTXOs for all users. Transactions between users are submitted to the contract by senders, and the contract then validates each transaction and updates the UTXO list. Users can exit by calling the contract, which then sums up the UTXOs belonging to that user and transfers the corresponding amount back to the user’s L1 wallet.

Strawman 2. The above construction suffers from two critical limitations. First, the contract must manage UTXOs for all users, requiring substantial onchain storage. Second, each transaction requires direct interaction with the contract, resulting in low throughput and high transaction fees.

To tackle these issues, we introduce minimal centralization by offloading UTXO management to the aggregator. Specifically, the aggregator becomes responsible for collecting transactions from users, validating them, and updating the UTXO list accordingly. The aggregator only needs to post data on L1 for a group of transactions rather than for each transaction, and this is done by sending the contract a block containing a short string that identifies the UTXO list.

To prevent the malicious aggregator from manipulating the UTXO list, we instantiate the identifier for the UTXO list as the root of a Merkle tree built from the UTXOs, and require the aggregator to generate a *block proof* that the UTXO tree root is correctly updated according to the valid transactions processed during block production.

Moreover, to allow users to exit the L2, aggregator generates a *balance proof* for each user, which attests to the user’s balance on L2 with respect to the latest block. By submitting the balance proof to the contract, users can demonstrate their balance and withdraw the funds.

Strawman 3. While the previous construction eliminates onchain UTXO storage and reduces per-transaction data on L1, it introduces a critical single point of failure: if the aggregator is unavailable or refuses to provide balance proofs, users cannot withdraw their funds and exit L2. Additionally, generating balance proofs for each user creates computational overhead for the aggregator.

We observe that it is possible to address both issues by leveraging client-side proving. Now, instead of relying on the aggregator for balance proofs, users maintain their own balance proofs locally. Although this increases computational overhead for users, we leverage IVC to enable efficient updates of balance proofs with minimal cost. This approach ensures users can always exit L2 by submitting their balance proof to the contract, regardless of the presence of the aggregator, and at the same time the aggregator’s burden is significantly reduced.

Final Construction. However, recall that balance proofs are associated with the latest block (i.e., the UTXO tree root). To update these proofs, users must first receive Merkle paths from the aggregator, but nothing guarantees the

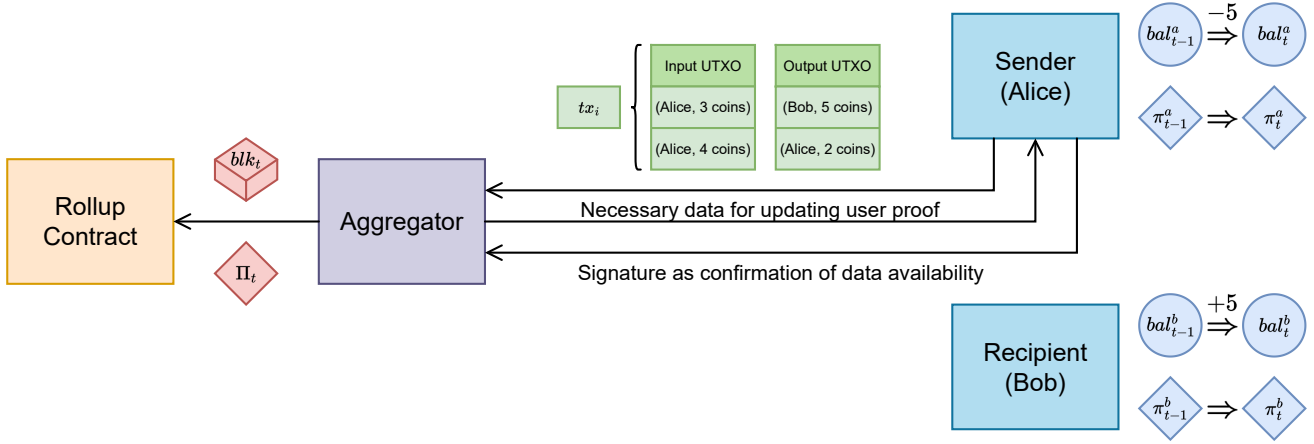


Figure 1: Overview of PlasmaFold. Here we consider two parties, the sender Alice and the recipient Bob, where Alice has two UTXOs with 3 and 4 coins respectively and wants to send 5 coins to Bob.

aggregator will make such data available. If the aggregator withholds Merkle paths, recipients cannot update their balance proofs even when transactions are reflected in the updated UTXO tree.

Our final construction enforces data availability through several modifications. The aggregator maintains an additional list of (unsettled) transactions and builds Merkle trees over these transactions. Client-side balance proofs now depend on the transaction tree root rather than the UTXO tree root, and the provision of Merkle paths in the transaction tree must be confirmed by users.

In order to guarantee users data availability, we leverage Intmax’s [27] setup, a straightforward construction guaranteeing the aggregator to only process transactions whose senders acknowledge that all necessary data has been made available to them. Let us recall how this setup works:

1. If the aggregator sends Merkle paths to users after receiving their transactions, users will acknowledge data availability by returning a signature on both their transaction and the transaction tree root to the aggregator: the aggregator will update the UTXO tree solely for signed transactions, guaranteed by the block proof.
2. If the aggregator fails to provide Merkle paths for some transactions, the corresponding signatures will be missing, and hence the consumption and creation of UTXOs for these transactions will not be reflected in the UTXO tree, thereby resolving any inconsistency between balance proofs and the UTXO tree.

In Figure 1, we provide a high-level overview of PlasmaFold’s transaction flow, which works as follows:

1. When transferring funds to other parties, each user first sends the transaction to the aggregator.

2. After receiving transactions from users, the aggregator
 - (a) checks validity of each transaction,
 - (b) builds a Merkle tree for transactions,
 - (c) sends necessary data for updating balance proof (e.g., Merkle paths) to each user,
 - (d) receives signatures from users,
 - (e) builds a Merkle tree for UTXOs based on the signed transactions,
 - (f) generates a *block proof* demonstrating that all previous steps are honestly done, and finally,
 - (g) posts the block together with the proof on L1.
3. The rollup contract verifies the proof and updates the L1 state according to the block if it is valid.
4. After the block has been accepted by the rollup contract, each user updates their own balance proof attesting to their balance on L2.

Further, as discussed above, the exit flow of PlasmaFold is simply the submission of the user's balance proof to the rollup contract, which verifies the proof and sends back the funds on L1.

2 Preliminaries

2.1 Incrementally Verifiable Computation

The notion of Incrementally Verifiable Computation (IVC) captures the idea of generating easy-to-verify proofs attesting to the correctness of a long-running, repetitive, computation [30]. We denote the repeatedly executed computation as *step function* $\mathcal{F}$. In the beginning, the prover $\mathcal{P}$ creates an initial *state* z_0 and a proof π_0 . Then, $\mathcal{P}$ repeatedly invokes $\mathcal{F}$, which consumes the current state z_i and produces the next state $z_{i+1} := \mathcal{F}(z_i; \text{aux}_i)$, where aux_i is some auxiliary information. After each invocation, $\mathcal{P}$ can use $\text{IVC}.\mathcal{P}$ to generate a proof π_{i+1} demonstrating that z_{i+1} is the state after $i + 1$ invocations of $\mathcal{F}$ starting from z_0 , with the help of the proof π_i attesting to z_i .

This notion is formalized as below.

Definition 1 (IVC). *An Incrementally Verifiable Computation (IVC) scheme is composed of four algorithms $\text{IVC} = (\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$:*

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$

The setup algorithm takes as input security parameter 1^λ and outputs the public parameters pp .

- $\mathcal{K}(\text{pp}, \mathcal{F}) =: (\text{ak}, \text{vk})$

The key generation algorithm takes as input public parameters pp and step function $\mathcal{F}$, which is computable in polynomial time. It outputs a pair (ak, vk) , where ak is the arguing key, and vk is the verification key.

- $\mathcal{P}(\mathbf{ak}, (i, \mathbf{z}_0, \mathbf{z}_i), \mathbf{aux}_i, \pi_i) \rightarrow \pi_{i+1}$

The proof generation algorithm takes as input arguing key $\mathbf{ak}$, index i , initial state $\mathbf{z}_0$, current state $\mathbf{z}_i$ in the i -th iteration, non-deterministic advice $\mathbf{aux}_i$, and proof π_i attesting to the honest derivation of $\mathbf{z}_i$. It outputs a new proof π_{i+1} attesting to the honest derivation of the next state $\mathbf{z}_{i+1} := \mathcal{F}(\mathbf{z}_i; \mathbf{aux}_i)$.

- $\mathcal{V}(\mathbf{vk}, (i, \mathbf{z}_0, \mathbf{z}_i), \pi_i) =: b$

The verification algorithm takes as input verification key $\mathbf{vk}$, index i , initial state $\mathbf{z}_0$, current state $\mathbf{z}_i$ in the i -th iteration, and proof π_i attesting to the honest derivation of $\mathbf{z}_i$. It outputs a bit b , indicating whether the proof is valid.

An IVC scheme IVC should be complete, knowledge sound, and succinct:

COMPLETENESS. (Perfect) completeness holds if for any Probabilistic Polynomial Time (p.p.t.) adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \mathcal{G}(\lambda), \\ (\mathcal{F}, i, \mathbf{z}_0, \mathbf{z}_i, \mathbf{aux}_i, \pi_i) \leftarrow \mathcal{A}(\text{pp}) \\ (\mathbf{ak}, \mathbf{vk}) := \mathcal{K}(\text{pp}, \mathcal{F}) \\ \mathbf{z}_{i+1} := \mathcal{F}(\mathbf{z}_i; \mathbf{aux}_i) \\ \mathcal{V}(\mathbf{vk}, (i, \mathbf{z}_0, \mathbf{z}_i), \pi_i) = 1 \\ \pi_{i+1} \leftarrow \mathcal{P}(\mathbf{ak}, (i, \mathbf{z}_0, \mathbf{z}_i), \mathbf{aux}_i, \pi_i) \end{array} : \mathcal{V}(\mathbf{vk}, (i+1, \mathbf{z}_0, \mathbf{z}_{i+1}), \pi_{i+1}) = 1 \right] = 1$$

KNOWLEDGE SOUNDNESS¹. Statistical (or computational) knowledge soundness holds if for every unbounded (or p.p.t.) adversary $\mathcal{A}$, there exists a p.p.t. extractor Ext such that for all input randomness r ,

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \mathcal{G}(\lambda), \\ (\mathcal{F}, (i \geq 1, \mathbf{z}_0, \mathbf{z}_i), \pi_i) := \mathcal{A}(\text{pp}; r) \\ (\mathbf{ak}, \mathbf{vk}) := \mathcal{K}(\text{pp}, \mathcal{F}) \\ \mathcal{V}(\mathbf{vk}, (i, \mathbf{z}_0, \mathbf{z}_i), \pi_i) = 1 \\ (\mathbf{z}_{i-1}, \mathbf{aux}_{i-1}, \pi_{i-1}) := \text{Ext}(\text{pp}; r) \end{array} : \begin{array}{l} \mathbf{z}_i = \mathcal{F}(\mathbf{z}_{i-1}; \mathbf{aux}_{i-1}) \\ \mathcal{V}(\mathbf{vk}, (i-1, \mathbf{z}_0, \mathbf{z}_{i-1}), \pi_{i-1}) = 1 \end{array} \right] \geq 1 - \varepsilon(\lambda)$$

SUCCINCTNESS. Succinctness holds if the size of π_i and the run time of $\mathcal{P}$ and $\mathcal{V}$ are independent of the number of iterations.

Note that, unlike the definition of succinctness in Succinct Non-Interactive Arguments of Knowledge (SNARKs), a succinct IVC may have proof size and verifier time that are linear in the size of $\mathcal{F}$. In addition, an IVC is not necessarily zero-knowledge.

To achieve a fully succinct and zero-knowledge IVC, one can include an additional zero-knowledge SNARK (zkSNARK) that compresses the proof while hiding the witnesses [?, 21]. This concept is formalized as *decider*, where we prove the satisfiability of a relation R^{Decider} that encodes the

Definition 2 (Decider). For an IVC scheme IVC , a step function $\mathcal{F}$, and a pair of arguing and verification key $(\mathbf{ak}_\Phi, \mathbf{vk}_\Phi) \leftarrow \text{IVC}.\mathcal{K}(\text{pp}, \mathcal{F})$, a decider $\text{Decider} = (\mathcal{K}, \mathcal{P}, \mathcal{V})$ is a zkSNARK for R^{Decider} . Given statement $\mathbf{x} = (k, \mathbf{z}_0, \mathbf{z}_k)$ and witness $\mathbf{w} = (\pi_k)$, $R^{\text{Decider}}(\mathbf{x}, \mathbf{w}) = 1$ if and only if

$$\text{IVC}.\mathcal{V}(\mathbf{vk}_\Phi, (k, \mathbf{z}_0, \mathbf{z}_k), \pi_k) = 1$$

¹We adopt [?]'s definition of knowledge soundness, which implies the notions in [?, ?, 21] that require Ext to extract all previous auxiliary values.

To realize IVC, one could adopt the stance taken from early approaches which consists in recursively proving a SNARKs, attesting to both the honest execution of the current step and the validity of the previous proof [5, 6]. In recent years, another line of work has focused on developing IVC based on *folding schemes* [20, 21, 32]. This approach consists into reducing the task of checking two or more instances of some relation to a single one [7, 21]. The advantage of such an approach lies in avoiding to compute a SNARK altogether, while keeping the recursion overhead (the price to pay for an iteration of the recursive protocol) low. For instance, Nova realizes IVC by instantiating a folding scheme for R1CS circuits over a cycle of elliptic curves [21]. More recently, Nebula introduced a folding schemes based IVC prover for proving long running virtual machine executions, a topic of particular interest for building verifiable rollups [3].

2.2 Accumulators, Vector Commitments, and Merkle Trees

Given a set of elements, *accumulators* can produce a short representation of the set. Membership of a set element can be verified without knowing the entire set, but only its representation.

Definition 3 (Accumulator). *An accumulator Acc consists of a tuple of algorithms $\text{Acc} = (\mathcal{G}, \mathcal{A}, \mathcal{U}^+, \mathcal{U}^-, \mathcal{P}, \mathcal{V})$:*

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$
The setup algorithm takes as input security parameter 1^λ and outputs the public parameters pp .
- $\mathcal{A}(\text{pp}, \mathbf{S}) \rightarrow \mathbf{A}$
The set accumulation algorithm takes as public parameters pp and a set $\mathbf{S} = \{v_i\}_{i=0}^{|\mathbf{S}|-1}$. It outputs an accumulator $\mathbf{A}$ that represents $\mathbf{S}$.
- $\mathcal{U}^+(\text{pp}, \mathbf{A}, \mathbf{S}, x) \rightarrow \mathbf{A}'$
The element addition algorithm takes as input public parameters pp , an accumulator $\mathbf{A}$, the set $\mathbf{S}$ it represents, and an element x to be added. It outputs an updated accumulator $\mathbf{A}'$ that represents $\mathbf{S}' := \mathbf{S} \cup \{x\}$.
- $\mathcal{U}^-(\text{pp}, \mathbf{A}, \mathbf{S}, x) \rightarrow \mathbf{A}'$
The element deletion algorithm takes as input public parameters pp , an accumulator $\mathbf{A}$, the set $\mathbf{S}$ it represents, and an element x to be deleted. It outputs an updated accumulator $\mathbf{A}'$ that represents $\mathbf{S}' := \mathbf{S} \setminus \{x\}$.
- $\mathcal{P}(\text{pp}, \mathbf{A}, \mathbf{S}, x) \rightarrow \varpi$
The membership proof generation algorithm takes as input public parameters pp , an accumulator $\mathbf{A}$, the set $\mathbf{S}$ it represents, and an element x . It outputs a membership proof ϖ for $x \in \mathbf{S}$.
- $\mathcal{V}(\text{pp}, \mathbf{A}, \varpi, x) =: b$
The membership proof verification algorithm takes as input public parameters pp , an accumulator $\mathbf{A}$, a membership proof ϖ , and an element x . It outputs a bit b indicating whether the proof is valid.

Vector commitments are a cryptographic primitive for a similar goal. The difference is that, while accumulators are designed for *set* membership, vector commitments can demonstrate that a value is at a specific *index* of some *vector*.

Definition 4 (Vector commitment). A vector commitment VC consists of a tuple of algorithms $\text{VC} = (\mathcal{G}, \mathcal{C}, \mathcal{U}, \mathcal{P}, \mathcal{V})$:

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$

The setup algorithm takes as input security parameter 1^λ and outputs the public parameters pp .

- $\mathcal{C}(\text{pp}, \mathbf{v}) \rightarrow \text{cm}$

The vector commitment algorithm takes as public parameters pp and a vector $\mathbf{v} = (v_i)_{i=0}^{|\mathbf{v}|-1}$. It outputs a commitment cm that represents $\mathbf{v}$.

- $\mathcal{U}(\text{pp}, \text{cm}, \mathbf{v}, x, \text{pos}) \rightarrow \text{cm}'$

The element updating algorithm takes as public parameters pp , a commitment cm , the vector $\mathbf{v}$ it represents, a value x , and an index pos . It outputs an updated commitment cm' that represents $\mathbf{v}' = (v'_i)_{i=0}^{|\mathbf{v}'|-1}$, where

$$v'_i = \begin{cases} v_i, & i \neq \text{pos} \\ x, & i = \text{pos} \end{cases}.$$

- $\mathcal{P}(\text{pp}, \text{cm}, \mathbf{v}, x, \text{pos}) \rightarrow \varpi$

The membership proof generation algorithm takes as input public parameters pp , a commitment cm , the vector $\mathbf{v}$ it represents, an element x , and its index pos . It outputs a membership proof ϖ for $x = v_i$.

- $\mathcal{V}(\text{pp}, \text{cm}, \varpi, x, \text{pos}) \rightarrow b$

The membership proof verification algorithm takes as input public parameters pp , a commitment cm , a membership proof ϖ , an element x , and its index pos . It outputs a bit b indicating whether the proof is valid.

Merkle trees are a concrete construction of accumulators and vector commitments based on hash functions. In a Merkle tree, the set (or vector) elements are treated as *leaves* of the tree, and the tree *root* r succinctly represents all the leaves. A membership proof of a Merkle tree, dubbed *Merkle path*, consists of the neighbors of nodes on the path from the leaf of interest to the root. In the rest of this paper, we explicitly instantiate accumulators and vector commitments as Merkle trees, which we denote as $\text{MT} = (\mathcal{G}, \mathcal{A} = \mathcal{C}, \mathcal{U}^+, \mathcal{U}^-, \mathcal{U}, \mathcal{P}, \mathcal{V})$ with $\text{pp} = \perp$ and $\mathbf{A} = \text{cm} = r$. Note that this instantiation is for notational convenience and does not affect the generality of our construction, as one can replace Merkle trees with any other accumulators and vector commitments.

3 Construction of PlasmaFold

This section presents the detailed construction of PlasmaFold. By leveraging Intmax2's short-id setup and data availability procedure [27] and designing an efficient IVC prover for both the aggregator and users, we obtain a scalable L2 with no user online requirements along with concretely RAM and speed efficient

server-side and client-side provers, featuring instant synchronization to the chain tip. We begin by describing the core concepts, data structures, and roles within PlasmaFold, followed by the protocol’s algorithmic specifications.

3.1 Notions and Data Structures

Height. The height t is a counter that tracks the number of blocks produced since the genesis block (where $t = 0$).

Keys. Every user in PlasmaFold has a secret key $\mathbf{sk}$ and a public key $\mathbf{pk}$. Upon joining the L2, a user generates a key pair $(\mathbf{sk}, \mathbf{pk})$ (which is not necessarily the same as the secret-public key pair of their L1 account) and submits the public key $\mathbf{pk}$ to the rollup contract. The contract then appends $\mathbf{pk}$ to a list of public keys maintained in its state.

Since the rollup contract does not sign transactions, we assign it a dummy public key $\mathbf{pk}^{\text{rollup}}$, allowing it to be treated as a standard user for deposits and withdrawals.

User IDs. We leverage Intmax2’s short-id setup [27], thereby minimizing the size of data posted onchain. This construction consists in replacing users’ public keys of users with short IDs, denoted as id . The rollup contract then records a mapping between the public keys and their corresponding id in its state, which is updated whenever a new user enrolls into PlasmaFold.

Nonces. Alongside the list of public keys, the rollup contract stores a list of nonces. For a user with public key $\mathbf{pk}$, the associated nonce $nonce$ is an increment only counter that tracks the total number of transactions signed with the corresponding secret key $\mathbf{sk}$. This mechanism ensures that a user’s balance proof correctly reflects all transactions they have initiated.

UTXOs and transactions. PlasmaFold operates in the UTXO model. A UTXO, denoted as u , is a tuple $(\mathbf{pk}, a)$, where a is the value and $\mathbf{pk}$ is the public key of the owner.

PlasmaFold supports three transaction types: *deposits*, *transfers*, and *withdrawals*. All transactions follow a general structure: $tx = ((u_i^{\mathcal{I}})_{i=0}^{m-1}, (u_i^{\mathcal{O}})_{i=0}^{n-1})$, where $(u_i^{\mathcal{I}})$ are the input UTXOs and $(u_i^{\mathcal{O}})$ are the output UTXOs. This definition captures general m -to- n transfers, with deposits and withdrawals being special cases that we will discuss below.

UTXOs and transactions are managed by the aggregator, but in distinct ways:

- Transactions are maintained on a per-block basis. At each height t , the aggregator collects a list of transactions, which is discarded at the beginning of the next block in favor of a new list.
- UTXOs are tracked persistently across all blocks. The aggregator maintains a comprehensive list of all unspent UTXOs, which is initialized at the genesis block and updated at each subsequent block to reflect newly created and spent UTXOs.

To minimize the on-chain data footprint, Merkle trees are built for both lists. The aggregator maintains a UTXO tree $\mathcal{T}_t^{\text{utxo}}$, and a transaction tree $\mathcal{T}_t^{\text{tx}}$. Their respective roots, r_t^{utxo} and r_t^{tx} , are included in block blk_t and posted on L1.

Deposits and withdrawals. As discussed above, deposits and withdrawals are regarded as two special cases of transactions.

More specifically, if a user wants to deposit L1 coins in exchange for L2 coins, they first stake the L1 coins to the rollup contract. The contract will update the user’s *pending deposit*, and a corresponding transaction will be generated by the aggregator, where the sender (of L2 coins) is the rollup contract and the recipient is the user. After the aggregator processes the transaction, a *deposit result* will be returned to the rollup contract, which will then decrease the pending deposit amount.

To withdraw funds, the user first sends a transaction to the aggregator, whose sender (of L2 coins) is the user and the recipient is the rollup contract. Then the aggregator processes the transaction in the same way as a transfer, but additionally sends the *withdrawal request* to the rollup contract. Eventually, the contract returns the corresponding L1 coins to the user.

To summarize, for deposits, we have $m = n = 1$, $u_0^{\mathcal{I}}.\text{pk}$ being the rollup contract’s public key, and $u_0^{\mathcal{O}}.\text{pk}$ being the user’s public key. For withdrawals, we have $n = 1$ (m can be greater than 1 so users can withdraw multiple UTXOs in one transaction), every $u_i^{\mathcal{I}}.\text{pk}$ being the user’s public key, and $u_0^{\mathcal{O}}.\text{pk}$ being the rollup contract’s public key.

We now define the deposit result and withdrawal request, both of which are generated by the aggregator and sent to the rollup contract.

A *deposit result* is a tuple (id, a) where id is the corresponding ID of $u_0^{\mathcal{O}}.\text{pk}$ and a is $u_0^{\mathcal{I}}.a$. It signifies that the aggregator has processed the deposit transaction, and the rollup contract should update its state accordingly.

A *withdrawal request* is a tuple (id, a) where id is the corresponding ID of $u_0^{\mathcal{I}}.\text{pk}$ and a is $u_0^{\mathcal{O}}.a$. It informs the rollup contract that the user has requested a withdrawal of a L1 coins, and the contract should return the corresponding amount to the user.

We denote the list of deposit results at block t as $\mathcal{L}_t^{\text{deposit}}$ and the list of withdrawal requests as $\mathcal{L}_t^{\text{withdraw}}$.

Signatures and signers. In order to guarantee data availability for transaction senders, thereby removing any online requirements, we leverage Intmax’s [27] setup. Roughly, this setup ensures that a transaction initiated by a user is processed and lands on the L1 after the user provided a signature over a piece of data acknowledging to his data availability and attesting to the correct inclusion of the transaction within a transaction batch. In the case of PlasmaFold, this signature consists in users signing the concatenation of the transaction itself with the aggregator-proposed transaction tree root. Transactions initiated by the rollup contract (i.e., deposits) do not require signatures, because their validity only depends on whether the corresponding amount of L1 tokens has been sent to the rollup contract.

Given transactions in a specific block, it then derives a list of signers broadcasted on the L1. To decrease onchain data requirements, this list of signers is posted in a compressed form, according to the short-id trick described in 3.1. In

the case of PlasmaFold, each element is either $\mathbf{pk}$ (i.e., the user public key that has signed the corresponding transaction), $\mathbf{pk}^{\text{rollup}}$ (i.e., the rollup contract public key), or $\perp$ (if the transaction is not a deposit but the signature is missing)

Similar to UTXOs and transactions, we build a Merkle tree $\mathcal{T}_t^{\text{signer}}$ for the signer list as well. However, the purpose of the signer tree is not to reduce the onchain data size, as the entire signer list will be posted onchain. Instead, the signer tree is built to allow the users to show in their balance proofs that their public key is in the signer list, thereby proving that they have signed the transaction tree root.

In the L2 block blk_t , we include the root of the $\mathcal{T}_t^{\text{signer}}$ as well as the signer list $\mathcal{L}_t^{\text{signer}}$.

Blocks. We summarize the contents of a (L2) block blk_t as follows:

$$blk_t = (t, r_t^{\text{utxo}}, r_t^{\text{tx}}, r_t^{\text{signer}}, \mathcal{L}_t^{\text{signer}}, \mathcal{L}_t^{\text{deposit}}, \mathcal{L}_t^{\text{withdraw}})$$

Looking ahead, in our concrete implementation, using Intmax2’s short-id trick [27], we replace the public keys in $\mathcal{L}_t^{\text{signer}}$ with their corresponding indexes in the public key list stored in the rollup contract, in order to further decrease PlasmaFold block size.

To allow users to generate balance proofs with respect to a certain block, we compute the hash of the block and build a block tree $\mathcal{T}_t^{\text{blk}}$. Specifically, in blk_t , only the parts that are necessary for the balance proof are treated as the preimage of the hash function, as defined below:

$$h_t^{\text{blk}} = H(t, r_t^{\text{tx}}, r_t^{\text{signer}})$$

3.2 Roles

In this section, we describe the three roles that make up PlasmaFold’s main actors. We first make an overview of users, in charge of producing their balance proof π . We then describe the aggregator, which verifiably builds blk_t using an IVC block proof Π_t . Finally, we touch on the data structures stored by the rollup contract on the L1.

Users. In PlasmaFold, users can perform the following tasks:

- Enroll in, stake funds to, and exit from PlasmaFold.
- Make transactions with each other and inform the aggregator of their transactions.
- Maintain balance proofs that attest to their balance amounts and their current transaction nonces.

They run their own IVC prover locally on their device, which, on input the previous proof π_t , executes a small step circuit $\mathcal{F}_{\text{user}}$ that updates the balance according to the transactions they sent or received at any particular blk_t , producing a new IVC proof π_{t+1} .

- Acknowledge the receipt of data required for their balance proofs from the aggregator and forward such data to the recipients of their transactions.

Looking ahead, the step circuit $\mathcal{F}_{\text{user}}$ takes as witness the Merkle path for the transaction that triggers the balance update, and the acknowledgment is done by returning the signature on the transaction tree root and their transactions to the aggregator.

Aggregator. The aggregator in PlasmaFold acts as an intermediary between users and the rollup contract, and is in charge of the following tasks:

- Collect transactions from users and propose blocks to the rollup contract.
- Maintain block proofs that attest to the validity of the blocks it produces.

This ensures that the aggregator correctly computes the state transition function of PlasmaFold, in a similar vein to IVLS [9]. To this end, the aggregator runs an IVC prover as well, which incrementally updates the state for each transaction. On the previous proof $\Pi_{t,i-1}$ input, the prover executes a step circuit $\mathcal{F}_{\text{agg}}$ that checks the i -th transaction, removes consumed UTXOs, and adds created UTXOs accordingly, producing a new IVC proof $\Pi_{t,i}$.

Note that unlike users in PlasmaFold, each of whom maintain the same balance proof throughout its entire life cycle (from join to exit), the aggregator discards the previous block proof and creates another one for the next block at each t . This is why we indicate both the block height t and the transaction index i in the block proof $\Pi_{t,i}$.

- Ensure data availability for users.

In line with the Intmax2 data availability procedure [27], the aggregator is responsible for returning the Merkle path for each transaction it processes to the transaction sender, so that the sender can be convinced that the transaction is included in the incoming block and can update their balance proof accordingly.

Rollup Contract. The rollup contract is deployed on an L1 blockchain. In its state, it records a table, where each entry contains a user’s ID, L1 address, public key, nonce, and pending deposits. Furthermore, it stores the root of the UTXO tree, the root of the block tree, and a list of block hashes. The rollup contract is responsible for the following tasks:

- Handle user enrollment.

An entry will be created when a user enrolls in PlasmaFold with their id, L1 address, and public key.

- Verify block proofs and update its state.

When the aggregator proposes a block, the rollup contract checks the validity of the associated block proof. If the verification passes, the contract updates its state with the new UTXO tree root and the new block tree root. It then appends the block hash to the list and increments the nonces of users who signed transactions in the block.

- Handle deposits and withdrawals.

For deposits, pending deposits in the contract's state will be increased for users who have staked funds on L1 to the contract, and they will be decreased when the aggregator collects the deposits and processes them in the proposed block. For withdrawals, the contract will send the requested amount of L1 coins to the user's L1 address.

- Handle user exists.

The rollup contract allows users to exit from PlasmaFold by submitting their balance proofs. For a valid balance proof, the contract will remove the user entry from its state and send the user's balance to their L1 address.

3.3 Main procedures

We now describe the main construction of PlasmaFold, which consists of the enrollment flow, staking flow, transaction flow, and exit flow.

3.3.1 Enrollment flow

To participate in PlasmaFold, a new user needs to follow the enrollment flow by interacting with the rollup contract, depicted in Figure 2.

1. The user calls the rollup contract by sending their L2 public key pk .
2. The rollup contract generates a new L2 ID id and records $id, addr, pk$ in its state, where $addr$ is the caller's L1 address.

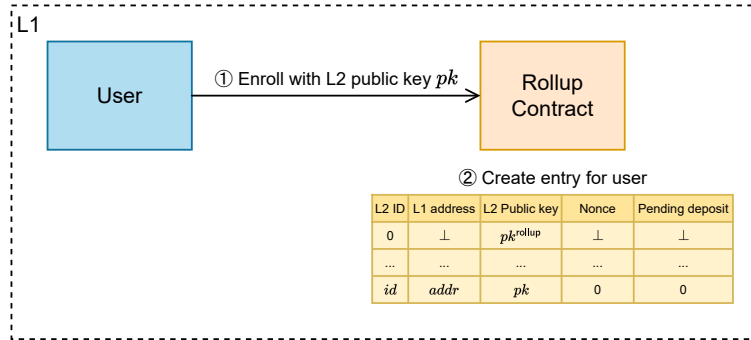


Figure 2: Enrollment flow in PlasmaFold

3.3.2 Staking flow

It is still impossible for users who newly enroll in PlasmaFold to make transactions, because they do not have any funds on the L2. Thus, they are required to exchange some of their L1 funds for L2 funds. This process involves two steps. Users first stake L1 funds to the rollup contract. Then, the aggregator creates deposit transactions on the L2 to transfer the staked funds to the corresponding L2 accounts. We describe the staking flow and illustrate it in Figure 3. Deposits handling is detailed right after.

1. The user transfers L1 funds with amount x to the rollup contract.
2. The rollup contract obtains $addr$ of the sender and increases the pending deposit for $addr$ by x .

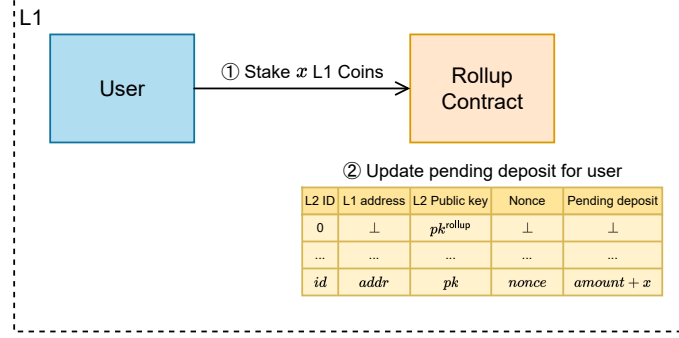


Figure 3: Staking flow in PlasmaFold

3.3.3 Transaction flow

Now, we discuss how transactions are processed in PlasmaFold. Recall that we consider three types of transactions: deposits, transfers, and withdrawals, where deposits and withdrawals can be treated as transfers from and to the rollup contract. This allows us to unify the logic for all transaction types and simplify the design of PlasmaFold.

Building transaction tree. The first step in processing transactions is to collect transactions and build the transaction tree. The aggregator collects transactions from both users and the rollup contract, where the latter provides pending deposits, while the former receives users transfers and withdrawals requests.

Following Intmax2’s data availability procedure [27], elements in the transaction tree are unsettled transactions: the inclusion of a transaction in the tree does not mean that the transaction is accepted because the aggregator may fail to guarantee the data availability to users, e.g. by not returning the Merkle paths for transactions. In this case, it is not reasonable to confirm all transactions. Otherwise, users who have not received the Merkle paths would be unable to update their balance proofs, even if their transactions have been confirmed.

1. The aggregator fetches pending deposits from the rollup contract, and for each non-zero pending deposit with amount a owned by pk , a transaction $tx := ((u_0^I), (u_0^O))$ is built, where $u_0^I = (pk^{\text{rollup}}, a)$, $u_0^O = (pk, a)$.
2. Every sender who wishes to transfer to some recipients or withdraw their funds, produces a transaction tx and informs the aggregator of tx .
3. The aggregator, after creating deposits and collecting transfers as well as withdrawals from users, builds the transaction tree $\mathcal{T}_t^{\text{tx}}$ on top of valid transactions.

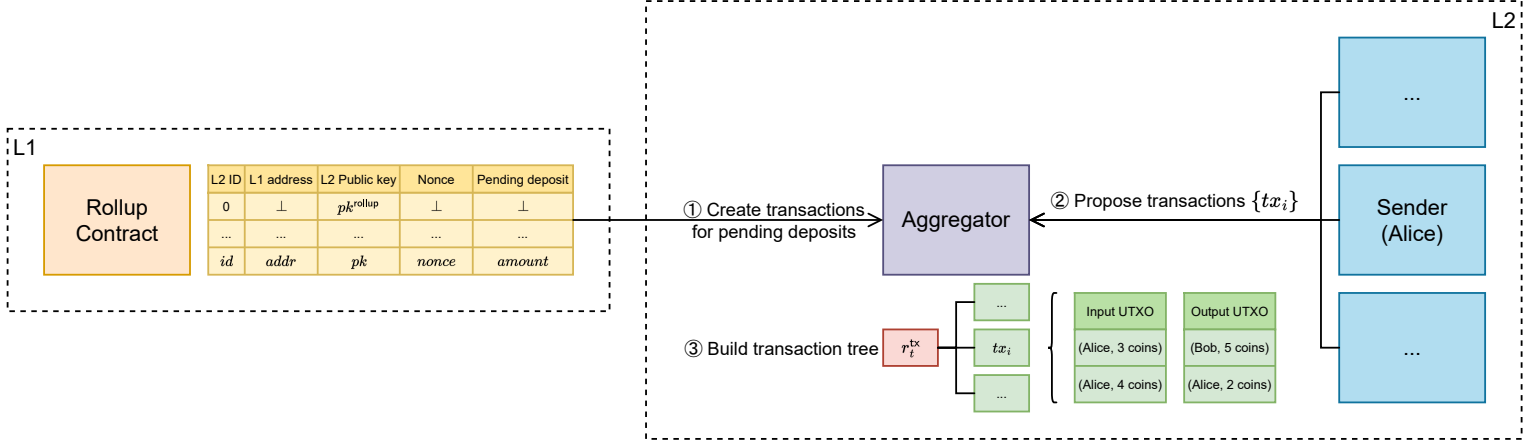


Figure 4: Transaction flow in PlasmaFold for building transaction tree

Building signer tree. Intmax2’s data availability procedure [27] requires the aggregator to send a Merkle path for each transaction to the sender, who will forward the path to the recipient and return a signature. Only transactions with valid signatures are processed by the aggregator. When landing the block on the L1, the aggregator broadcasts a list of signers to signal users who have attested, through this signature, to the availability of offchain data. In the context of PlasmaFold, the signature is on the concatenation of the transaction and the transaction tree root, preventing other users from replaying the transaction in a different block. In order to allow users to efficiently prove that their transaction has been signed in their balance proofs, we further build a Merkle tree on top of the signer list. Note that it is possible that a malicious aggregator refuses to include the corresponding user in the signer list even if the user has provided a signature. However, the transaction will not be accepted in this case, and the user can still exit PlasmaFold if they notice that their public key is missing from the signer list. In PlasmaFold, we limit one transaction per sender per block, so it is impossible to replay a transaction in the same block. One can extend PlasmaFold to allow senders to make multiple transactions in the same block by adding unique identifiers to transactions.

- For each non-deposit transaction tx_i , the aggregator sends back to the sender a Merkle path $\varpi_{t,i}^{\text{tx}}$ from tx_i to the transaction tree root r_t^{tx} , showing that tx_i is included in $\mathcal{T}_t^{\text{tx}}$. This Merkle path is necessary for the sender to update their balance proof.
- Every sender, after receiving the Merkle path, forwards it to the recipient of the transaction, allowing the recipient to update their balance proof as well.
- Every sender acknowledges the receipt of the Merkle path by generating a signature σ_i on the transaction tx_i and the transaction tree root r_t^{tx} , which is then returned to the aggregator.
- The aggregator verifies the signatures after collecting them from all senders.

8. The aggregator builds the signer tree $\mathcal{T}_t^{\text{signer}}$ on top of the list of public keys of senders who have provided valid signatures.

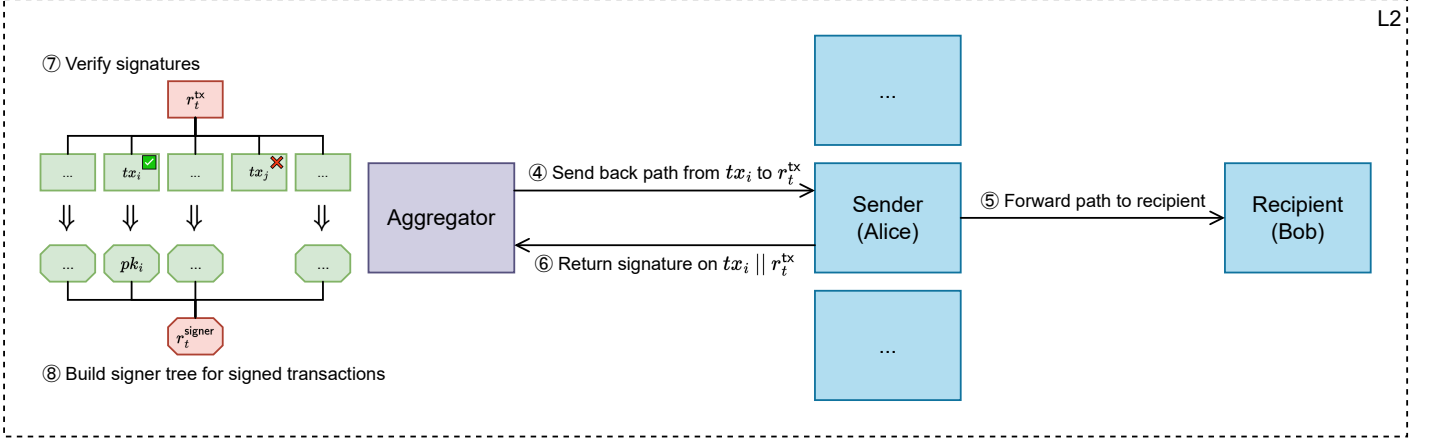


Figure 5: Transaction flow in PlasmaFold for building signer tree

Updating UTXO tree. For accepted transactions (i.e. those with valid signatures), the aggregator updates the UTXO tree to reflect the consumption of input UTXOs and the creation of output UTXOs. This ensures users cannot double spend their UTXOs, because the UTXO tree root only contains UTXOs that they currently own.

9. The aggregator updates the UTXO tree $\mathcal{T}_{t-1}^{\text{utxo}}$ to $\mathcal{T}_t^{\text{utxo}}$ by removing input UTXOs and inserting output UTXOs for all transactions with valid signatures.

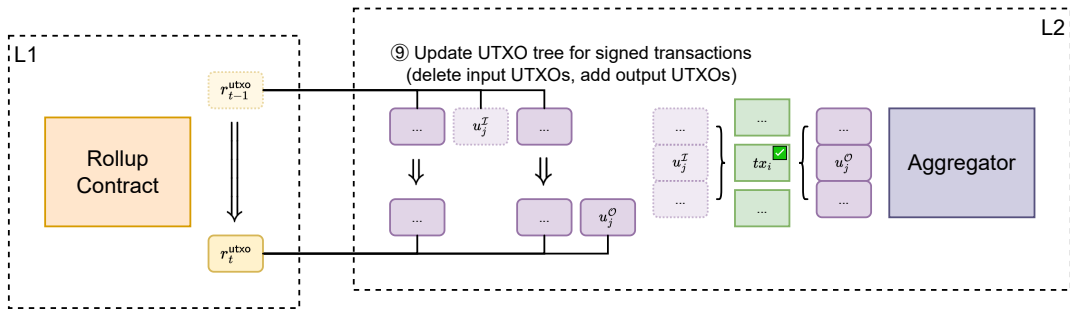


Figure 6: Transaction flow in PlasmaFold for updating utxo tree

Updating the block tree. Looking ahead, a user's balance proof shows that updates to the user balance are solely based on the transactions that the user

has sent or received. To ensure that the claimed transactions are indeed the ones included in some existing block (rather than deceptively created by the user), the circuit for the balance proof will require the transaction in every step to be contained in some block, and that block matches the one in the rollup contract's state.

A naive approach would be to introduce the chained hash of the blocks. For each block, both the rollup contract and the user would absorb the current block hash into an accumulated hash, and at the same time, the user would ensure not only the correct update of their balance in their balance proof, but also the honesty of chained hash computation. However, this would require the user to be consistently online and update their proof, even if they do not send or receive any transactions in some blocks.

We build a block tree $\mathcal{T}_t^{\text{blk}}$ in order to allow the user to prove the inclusion of blocks in the onchain state without requiring them to be online for irrelevant blocks. Since the leaves of $\mathcal{T}_t^{\text{blk}}$ (i.e., the block hashes) are public and stored onchain, a straightforward design is to ask the rollup contract to update the block tree. However, this would introduce a trade-off in terms of the onchain costs and client-side costs. If we use natively supported hash functions for the block tree, then the user circuit will become much larger. On the other hand, if we use a circuit-friendly hash function, then the rollup contract will consume more gas to update the block tree.

We tackle this trade-off by asking the aggregator to update the block tree, and the block proof now guarantees the correctness of this update. In this way, we can switch to circuit-friendly hash functions for smaller user circuits without increasing the onchain costs.

10. The aggregator computes the block hash value $h_t^{\text{blk}} = H(t, r_t^{\text{tx}}, r_t^{\text{signer}})$ and updates the block tree $\mathcal{T}_{t-1}^{\text{blk}}$ by inserting h_t^{blk} , producing the new block tree $\mathcal{T}_t^{\text{blk}}$.

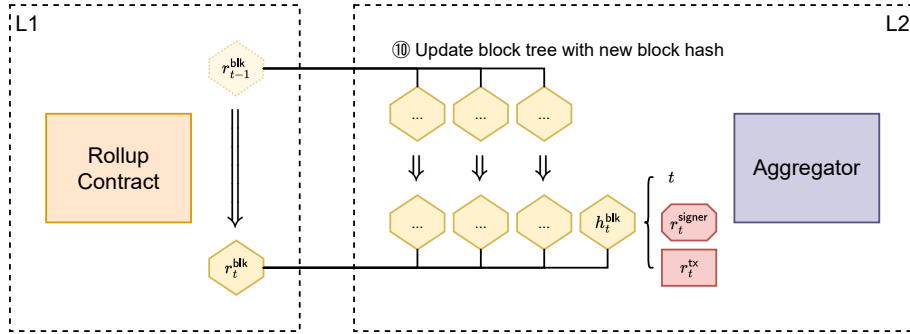


Figure 7: Transaction flow in PlasmaFold for updating block tree

Generating block proof. Now that all the block data is ready, the aggregator produces a block proof Π_t that attests to the validity of the block. It will trigger the update of the L1 rollup contract state.

11. The aggregator generates a block proof Π_t and proposes the block $blk_t := (t, r_t^{\text{utxo}}, r_t^{\text{tx}}, r_t^{\text{signer}}, \mathcal{L}_t^{\text{signer}}, \mathcal{L}_t^{\text{deposit}}, \mathcal{L}_t^{\text{withdraw}})$ to the rollup contract, together with the block proof Π_t , the block hash h_t^{blk} , and the new block tree root r_t^{blk} .
12. The rollup contract verifies the block proof Π_t against the block data and its current state. If the block is valid, then the contract updates its state by:
 - (a) incrementing the nonce for each signer in $\mathcal{L}_t^{\text{signer}}$,
 - (b) decreasing the pending deposits of depositors according to the deposit results,
 - (c) sending the corresponding amount of L1 coins to the withdrawers according to the withdrawal requests,
 - (d) updating the UTXO tree root to r_t^{utxo} ,
 - (e) updating the block tree root to r_t^{blk} , and
 - (f) appending the block hash h_t^{blk} to the list of block hashes.

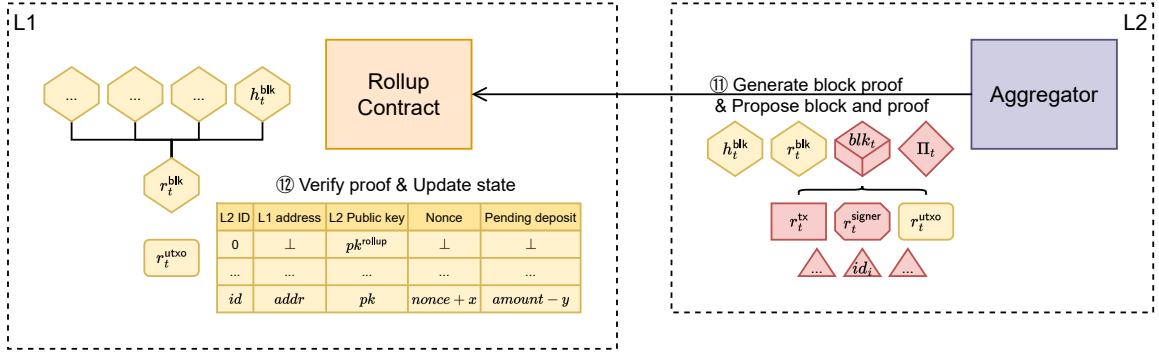


Figure 8: Transaction flow in PlasmaFold for generating and verifying the block proof

Updating balance proof. Finally, users update their balance proofs π_t accordingly. We introduced above the block tree, allowing users who do not send or receive transactions in the current block to be offline and skip the update.

To ensure the validity of a transaction and the provision of its Merkle path in the transaction tree, we require the transaction recipient to verify their balance proof's correctness, before providing any good or service that the sender has paid for. Otherwise, a malicious sender can either claim an invalid transaction that is not included in the block, or provide an invalid Merkle path.

13. Every sender or recipient, whose transaction is included in blk_t , updates their balance as well as the balance proof π_t using the public L1 data and the Merkle path provided by the aggregator.
14. Every recipient, after checking the validity of their updated balance proof, provides the sender with the goods or services they have purchased.

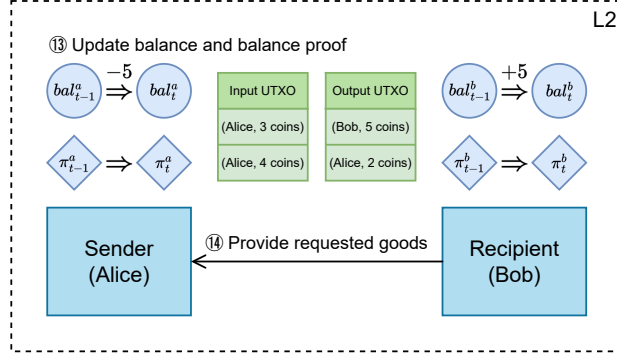


Figure 9: Transaction flow in PlasmaFold for updating the balance proof

3.3.4 Exit flow

PlasmaFold exit flow is non-interactive and instantaneous, providing users with sovereignty over their funds regardless of the aggregator’s availability. Users can exit PlasmaFold at any time without requiring the presence of the aggregator, simply by submitting their balance proofs to the rollup contract. Upon successful verification, they will receive their balance in L1 coins in the next L1 block.

The exit process follows a straightforward two-step procedure, which is also illustrated in Figure 10.

1. The user, at block t , calls the rollup contract with their balance and the corresponding balance proof π_t .
2. The rollup contract verifies the balance proof against the claimed balance and the current state. If the verification passes, the contract removes the user entry from its state and transfers the user’s balance to their L1 address.

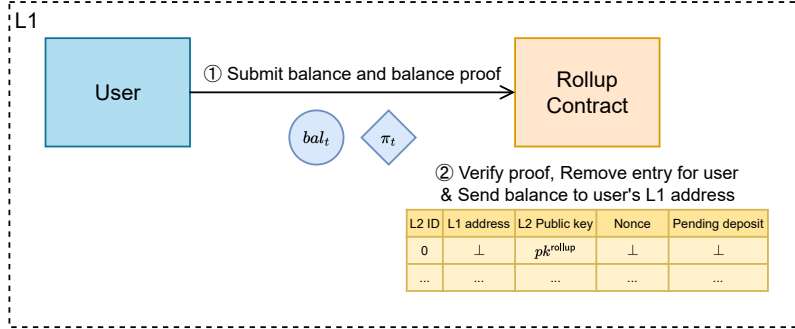


Figure 10: Exit flow in PlasmaFold

3.4 Block and Balance Proofs

Finally, we explain how the aggregator and users maintain their IVC proofs and how the rollup contract verifies them. For clarity, we only consider 1-to-1

transactions, but it is straightforward to extend our circuits to support m -to- n transactions. In addition, we will write $X_{t,i}$ to represent the state of X at the i -th transaction in block t . For instance, the state of the UTXO tree at block t , after having processed the i -th transaction is noted $\mathcal{T}_{t,i}^{\text{utxo}}$.

3.4.1 Generation and Verification of Block Proofs

For the aggregator, it is possible to build a single SNARK circuit that validates all transactions in a block. However, this would result in a large circuit size, leading to significant memory usage.

To reduce the burden on the aggregator, we build a step circuit $\mathcal{F}_{\text{agg}}$ that processes one transaction at a time, whose construction is depicted in Algorithm 1. With $\mathcal{F}_{\text{agg}}$, the aggregator can use IVC to incrementally prove the validity of each transaction in the block. Specifically, a new balance proof $\Pi_{t,0}$ is initialized at the beginning of each block t . Then the aggregator proves each transaction in the block sequentially, and in each step i , it updates $\Pi_{t,i}$ to $\Pi_{t,i+1}$ by running IVC.P . After processing all k transactions, the IVC proof becomes $\Pi_t := \Pi_{t,k}$, which is then sent to the rollup contract for verification.

Construction of aggregator circuit. The aggregator circuit $\mathcal{F}_{\text{agg}}$ essentially ensures the correct update of the transaction tree, the signer tree, and the UTXO tree.

More specifically, it first checks the validity of the transaction, ensuring that 1) the amount in the input UTXO is the same as that in the output UTXO, 2) the public key in the input UTXO is the same as the public key of the sender, and 3) the input UTXO is contained in the current UTXO tree.

Next, $\mathcal{F}_{\text{agg}}$ inserts the transaction into the transaction tree. It then verifies the signature on the transaction tree root and the transaction, ensuring that the non-deposit transaction is signed by the sender.

If the signature is valid, or the transaction is a deposit (i.e. the sender is the rollup contract), the circuit will update the UTXO tree by deleting the input UTXO and inserting the output UTXO (as long as these UTXOs are not owned by the rollup contract). In this case, the signer tree will be updated as well, and the position of the sender's public key in the signer tree will be the same as that of the corresponding transaction in the transaction tree.

In the above process, it is unnecessary for the circuit to update every intermediate nodes of the trees when performing insertion or deletion. Instead, the state $\mathbf{z}_i$ of the circuit contains roots of the trees $(r_{t,i}^{\text{utxo}}, r_{t,i}^{\text{tx}}, r_{t,i}^{\text{signer}})$, which are the only values that need to be updated in each step. Note that in the i -th step of the aggregator's IVC, the transaction tree only contains the first i transactions, but a valid signature provided by the sender is on the root $r_t^{\text{tx}} = r_{t,k}^{\text{tx}}$ of the transaction tree that contains all transactions. Therefore, we include the final transaction tree root r_t^{tx} in the state $\mathbf{z}_i$ as well. Unlike other tree roots that are modified in each step, r_t^{tx} remains unchanged throughout the entire block processing.

Optimized approach to handling public inputs. Recall that the aggregator needs to publish the lists of signers, deposit results, and withdrawal requests in the block, which are used to verify the block proof. If we include these public inputs in the state $\mathbf{z}_i$ of the aggregator's circuit, then the state size will

grow linearly with the number of transactions, which is suboptimal in terms of efficiency. In our optimized approach, we utilize hash chains to handle these lists. Specifically, the state z_i now additionally contains the accumulated hashes of the signers, deposit results, and withdrawal requests, denoted as $\sigma_{t,i}^{\text{signer}}$, $\sigma_{t,i}^{\text{deposit}}$, and $\sigma_{t,i}^{\text{withdraw}}$ respectively. In each step, the circuit accumulates the new elements of these lists into the corresponding accumulated hashes, thereby not increasing the size of the next state z_{i+1} .

Proving block tree update. A missing piece in $\mathcal{F}_{\text{agg}}$ is the update of the block tree. It is possible to directly add this check to $\mathcal{F}_{\text{agg}}$ by computing $r_t^{\text{blk}} \leftarrow \text{MT}.\mathcal{U}(r_{t-1}^{\text{blk}}, H(t, r_t^{\text{tx}}, r_t^{\text{signer}}), t)$ for every step i . However, this approach repeats exactly the same computation on the same data for k times, while the block tree only needs to be updated once.

A more efficient way to check this update is to utilize the flexible design of the decider SNARK. Recall that the decider SNARK is executed at the end of IVC to compress the IVC proof into a fully succinct one. By integrating the update of the block tree into the decider SNARK circuit, we can verify the correctness of the block tree update without repeating it in every incremental step.

Algorithm 1: Aggregator circuit. We highlight auxiliary **witness** values.

```

1 Circuit  $\mathcal{F}_{\text{agg}}(z_i)$ :
2    $(r_{t,i}^{\text{utxo}}, r_{t,i}^{\text{tx}}, r_t^{\text{tx}}, r_{t,i}^{\text{signer}}, \sigma_{t,i}^{\text{signer}}, \sigma_{t,i}^{\text{deposit}}, \sigma_{t,i}^{\text{withdraw}}) := z_i$ 
3    $(u^{\mathcal{I}}, u^{\mathcal{O}}) := \text{tx}$ 
4   enforce  $u^{\mathcal{I}}.a = u^{\mathcal{O}}.a$ 
5   enforce  $u^{\mathcal{I}}.pk = \text{pk}^{\text{sender}}$ 
6   enforce  $\text{MT}.\mathcal{V}(r_{t,i}^{\text{utxo}}, u^{\mathcal{I}}, \varpi^{\text{utxo}})$ 
7    $r_{t,i+1}^{\text{tx}} \leftarrow \text{MT}.\mathcal{U}(r_{t,i}^{\text{tx}}, \text{tx}, i)$ 
8    $r_{t,i+1}^{\text{utxo}} := r_{t,i}^{\text{utxo}}$ 
9   if  $\text{Sig}.\mathcal{V}(\text{pk}^{\text{sender}}, r_t^{\text{tx}} \parallel \text{tx}, \sigma) = 1 \vee \text{pk}^{\text{sender}} = \text{pk}^{\text{rollup}}$  then
10    if  $u^{\mathcal{I}}.pk = \text{pk}^{\text{rollup}}$  then
11       $\sigma_{t,i+1}^{\text{deposit}} \leftarrow H(\sigma_{t,i}^{\text{deposit}}, (u^{\mathcal{O}}.pk, u^{\mathcal{I}}.a))$ 
12    else
13       $r_{t,i+1}^{\text{utxo}} \leftarrow \text{MT}.\mathcal{U}^-(r_{t,i+1}^{\text{utxo}}, u^{\mathcal{I}})$ 
14    if  $u^{\mathcal{O}}.pk = \text{pk}^{\text{rollup}}$  then
15       $\sigma_{t,i+1}^{\text{withdraw}} \leftarrow H(\sigma_{t,i}^{\text{withdraw}}, (\text{pk}^{\text{sender}}, u^{\mathcal{O}}.a))$ 
16    else
17       $r_{t,i+1}^{\text{utxo}} \leftarrow \text{MT}.\mathcal{U}^+(r_{t,i+1}^{\text{utxo}}, u^{\mathcal{O}})$ 
18       $r_{t,i+1}^{\text{signer}} \leftarrow \text{MT}.\mathcal{U}(r_{t,i}^{\text{signer}}, \text{pk}^{\text{sender}}, i)$ 
19       $\sigma_{t,i+1}^{\text{signer}} \leftarrow H(\sigma_{t,i}^{\text{signer}}, \text{pk}^{\text{sender}})$ 
20    else
21       $\sigma_{t,i+1}^{\text{signer}} \leftarrow H(\sigma_{t,i}^{\text{signer}}, \perp)$ 
22  return  $z_{i+1} := (r_{t,i+1}^{\text{utxo}}, r_{t,i+1}^{\text{tx}}, r_t^{\text{tx}}, r_{t,i+1}^{\text{signer}}, \sigma_{t,i+1}^{\text{signer}}, \sigma_{t,i+1}^{\text{deposit}}, \sigma_{t,i+1}^{\text{withdraw}})$ 

```

Verification of block proof. After receiving the block proof, the rollup contract verifies it against the public inputs derived from the block data and the L1 state. Since we use hash chains to compress the public inputs, the rollup contract can replace expensive group operations on $\mathcal{L}_t^{\text{signer}}$, $\mathcal{L}_t^{\text{deposit}}$, and $\mathcal{L}_t^{\text{withdraw}}$ with cheap hash operations during the verification, as long as the hash function is instantiated with the ones supported by the blockchain execution environment (e.g. SHA256). That is, the contract now only needs to compute σ_t^{signer} , $\sigma_t^{\text{deposit}}$, and $\sigma_t^{\text{withdraw}}$ by respectively hashing $\mathcal{L}_t^{\text{signer}}$, $\mathcal{L}_t^{\text{deposit}}$, and $\mathcal{L}_t^{\text{withdraw}}$ in a chained fashion. Then, the proof is verified against the initial state $\mathbf{z}_0 = (r_{t-1}^{\text{utxo}}, \perp, r_t^{\text{tx}}, \perp, \perp, \perp, \perp)$ and the final state $\mathbf{z}_k = (r_t^{\text{utxo}}, r_t^{\text{tx}}, r_t^{\text{tx}}, r_t^{\text{signer}}, \sigma_t^{\text{signer}}, \sigma_t^{\text{deposit}}, \sigma_t^{\text{withdraw}})$.

3.4.2 Generation and Verification of Balance Proofs

Unlike the aggregator who discards the previous balance proof and starts with a new one at the beginning of each block, users keep their balance proofs throughout their entire life cycle in PlasmaFold, from enrollment to exit. IVC is used to incrementally update the balance proof from π_i to π_{i+1} , for each transaction that the user sends or receives. Here, the i -th step does not necessarily correspond to the block at height $t = i$, since our design allows users to skip blocks where they did not send or receive transactions, as discussed in Section 3.3.3. In other words, the block heights t_i and t_{i+1} at two consecutive IVC steps may satisfy $t_{i+1} \gg t_i$.

Construction of user circuit. The user circuit $\mathcal{F}_{\text{user}}$, as detailed in Algorithm 2, is designed to update the user's balance and nonce based on the i -th transaction that the user has sent or received.

Given a transaction tx (as witness), the circuit records in its state $\mathbf{z}_i$ the previous block height t_{i-1} and the position pos_{i-1} of the previous transaction in its transaction tree. In the i -th step, it first ensures that the pair of block height and transaction position (t_i, pos_i) is greater than the previous one (t_{i-1}, pos_{i-1}) . In other words, they should satisfy $t_i > t_{i-1}$ or $t_i = t_{i-1}$ and $pos_i > pos_{i-1}$. Then, the circuit ensures that pos_i is the position of tx in the transaction tree by running MT.V , with the Merkle path ϖ^{tx} received from the aggregator (if the user is the sender) or the sender (if the user is the recipient). It reproduces this check for the signer tree, guaranteeing that the transaction is signed by the sender, and the Merkle path ϖ^{signer} can be computed from the rollup contract's state.

The user circuit's state $\mathbf{z}_i$ additionally contains the user's balance bal_i and nonce $nonce_i$. They are updated according to the transaction tx . If the user is the sender, then the circuit deducts the amount in the input UTXO from bal_i and increments $nonce_i$. If the user is the recipient, the circuit adds the amount in the output UTXO to bal_i . To determine whether the user is sender or recipient, we further include the user's pk in the state $\mathbf{z}_i$, which is unchanged in each step and is only used to compare with the public keys contained in the input and output UTXOs.

Proving inclusion of blocks. For now, the user circuit only ensures that the transaction and the signer are in *some* Merkle trees with roots $r_{t_i}^{\text{tx}}$ and $r_{t_i}^{\text{signer}}$, but these roots are not necessarily the ones in the rollup contract's state.

To guarantee the honesty of these roots, we require the user circuit to commit to the block at each step i (i.e. the height t_i , the transaction tree root $r_{t_i}^{\text{tx}}$, and the signer tree root $r_{t_i}^{\text{signer}}$). When exiting PlasmaFold, the user modifies the decider SNARK circuit, which not only compresses the IVC proof, but also proves that all committed blocks involved in their IVC steps are contained in the latest block tree $\mathcal{T}_t^{\text{blk}}$, thereby preventing the user from claiming false blocks.

Similar to the aggregator circuit, we use chained hashes as well to commit to the list of blocks without increasing the size of the state z_i . In the user circuit, we include σ_i^{blk} in z_i as the accumulated hash of all blocks up to step i , and the circuit now additionally absorbs the current block hash in σ_i^{blk} . The decider SNARK circuit verifies that the commitment opens to all block hashes, and that these block hashes are leaves of the block tree $\mathcal{T}_t^{\text{blk}}$. By using circuit-friendly hash functions for commitment and block tree, we can make this side proof lightweight and efficient, without relying on the rollup contract to check the inclusion of blocks in the block tree.

Algorithm 2: User circuit. We highlight auxiliary [witness](#) values.

```

1 Circuit  $\mathcal{F}_{\text{user}}(z_i)$ :
2    $(bal_i, nonce_i, pk, t_{i-1}, pos_{i-1}, \sigma_i^{\text{blk}}) := z_i$ 
3   enforce  $t_i \geq t_{i-1}$ 
4   if  $t_i = t_{i-1}$  then
5     enforce  $pos_i > pos_{i-1}$ 
6   enforce  $\text{MT.V}(r_{t_i}^{\text{tx}}, tx, pos_i, \varpi^{\text{tx}})$ 
7   enforce  $\text{MT.V}(r_{t_i}^{\text{signer}}, u^{\mathcal{I}}.pk, pos_i, \varpi^{\text{signer}})$ 
8    $(u^{\mathcal{I}}, u^{\mathcal{O}}) := tx$ 
9    $bal_{i+1} := bal_i$ 
10   $nonce_{i+1} := nonce_i$ 
11  if  $u^{\mathcal{I}}.pk = pk$  then
12     $bal_{i+1} := bal_{i+1} - u^{\mathcal{I}}.a$ 
13     $nonce_{i+1} := nonce_{i+1} + 1$ 
14  if  $u^{\mathcal{O}}.pk = pk$  then
15     $bal_{i+1} := bal_{i+1} + u^{\mathcal{O}}.a$ 
16   $\sigma_{i+1}^{\text{blk}} \leftarrow H(\sigma_i^{\text{blk}}, H(t_i, r_{t_i}^{\text{tx}}, r_{t_i}^{\text{signer}}))$ 
17  return  $z_{i+1} := (bal_{i+1}, nonce_{i+1}, pk, t_i, pos_i, \sigma_{i+1}^{\text{blk}})$ 

```

Verification of balance proof. The rollup contract can verify the balance proof against the initial state $z_0 = (0, 0, pk, -1, 0)$, the final state $z_l = (bal_l, nonce, pk, t_l, pos_l)$, and the latest block tree root r_t^{blk} . Here, $nonce$ and pk are obtained from its own state, while bal_l , t_l , and pos_l are provided by the user as the final balance, the last block height, and the position of the last transaction in the transaction tree. Note that although the IVC state also contains the chained hash σ^{blk} , it is eliminated in the final public inputs due to the modified decider SNARK. Instead, an additional public input r_t^{blk} is introduced, ensuring that the decider circuit checks each block's membership in the block tree $\mathcal{T}_t^{\text{blk}}$.

3.5 Extensions

PlasmaFold can run in two modes, the hybrid mode and the validum mode. In the former case, users maintain their own balance proofs and run the IVC prover on their device as described above, while it is unnecessary for them to do so in the latter case. Therefore, the main advantage of running PlasmaFold in validum mode is to save local prover costs. Such a setting requires users and the aggregator to agree on a bespoke protocol for exits, where users need to obtain the necessary auxiliary data from the aggregator.

4 Implementation and Evaluation

We provide an open-source prototype implementation of PlasmaFold in Rust, under an MIT license regime². Our implementation contains PlasmaFold main data structures, the circuits for the aggregator and users, and a mock rollup contract for testing the interaction with an aggregator and users. Under the hood, we use the `Arkworks` [2] library for field and group operations as well as cryptographic primitives, and leverage the `Sonobe` [1] library for its modular and efficient implementation of folding-based IVC.

In our implementation, we instantiate IVC with Nova-based IVC [21], Sig with Schnorr signature, and the hash function in Merkle trees with Poseidon [16]. For the chained hash, one can choose between SHA256 and Poseidon, depending on the trade-off between the circuit size and onchain execution costs. We utilize the BN254-Grumpkin half-pairing cycle as the underlying elliptic curves for our IVC.

Furthermore, although both the aggregator circuit and the user circuit described in Section 3.4 handle one transaction per step, our implementation features transaction batching. This is beneficial because Nova-based IVC adds to step circuits a constant overhead for recursively verifying the previously folded instances, which can be amortized by processing multiple transactions in a single step.

Benchmarks. To demonstrate the performance of PlasmaFold and study how transaction batch size impacts efficiency, we evaluate the aggregator and user IVC by measuring the circuit size, prover time, RAM usage and SRS size. In our evaluation, we consider transactions with 4 input UTXOs and 4 output UTXOs, capturing a greater than average size of transactions in a blockchain like Bitcoin [12].

For the aggregator performance, we run the benchmarks on a Linux PC with an Intel Core i9-12900K CPU and 64GB of RAM. We consider two modes of the aggregator circuit: one that only considers transfers, and another that also supports deposits and withdrawals. Both modes use SHA256 as the hash function for the chained hash H . The results for the transfer-only mode and the full mode are shown in Table 1 and Table 2, respectively.

From the results, we observe that the aggregator circuit in transfer-only mode is $\sim 1.5\times$ more efficient than the full mode, because the latter additionally accumulates the list of deposits and withdrawals with SHA256 as H , which is expensive in arithmetic circuits. Also, the prover time per transaction can be

²Available at <https://github.com/dmpierre/plasma-fold>

Batch size	$ \mathcal{F}_{\text{agg}} $	$ \mathcal{F}_{\text{agg}}^{\text{aug}} $	IVC. $\mathcal{P}$ (s)	RAM usage (MB)	$ \text{srs} $ (MB)
1	257k	321k	0.40	2533	313
2	520k	585k	0.69	4108	579
4	1045k	1113k	1.23	7841	1111
8	2096k	2169k	2.35	15215	2175
16	4198k	4282k	4.57	30520	4304

Table 1: Benchmarking results of aggregator circuit in transfer-only mode.

Batch size	$ \mathcal{F}_{\text{agg}} $	$ \mathcal{F}_{\text{agg}}^{\text{aug}} $	IVC. $\mathcal{P}$ (s)	RAM usage (MB)	$ \text{srs} $ (MB)
1	659k	733k	0.55	4268	437
2	1334k	1409k	0.99	8050	827
4	2684k	2762k	1.84	15889	1607
8	5383k	5467k	3.54	31493	3168
16	10783k	10877k	7.02	62063	6288

Table 2: Benchmarking results of aggregator circuit in full mode.

slightly reduced with small batch sizes (e.g., 2 or 4), but larger batch sizes lead to marginal improvements.

For the user performance, we compile our implementation to WebAssembly and perform the tests within the Chrome web browser of version 137.0.7151.57 on an Apple Macbook M2 Pro, since most popular blockchain wallets run in browsers. Table 3 and Table 4 summarize our results, where SHA256 and Poseidon are used for the chained hash H , respectively.

Batch size	$ \mathcal{F}_{\text{user}} $	$ \mathcal{F}_{\text{user}}^{\text{aug}} $	IVC. $\mathcal{P}$ (s)	RAM usage (MB)	$ \text{srs} $ (MB)
1	94k	150k	3.64	441	97
2	111k	167k	4.25	490	114
5	161k	218k	5.70	653	164
8	212k	268k	7.34	908	232
10	246k	302k	8.42	1050	266

Table 3: Benchmarking results of user circuit with SHA256 as H for chained hash.

Batch size	$ \mathcal{F}_{\text{user}} $	$ \mathcal{F}_{\text{user}}^{\text{aug}} $	IVC. $\mathcal{P}$ (s)	RAM usage (MB)	$ \text{srs} $ (MB)
1	19k*	76k	2.77	291	73
2	36k*	92k	3.32	343	90
5	87k	143k	4.85	579	149
8	138k	193k	6.40	737	200
10	171k	227k	7.33	845	233

Table 4: Benchmarking results of user circuit with Poseidon as H for chained hash. We indicate with * all circuits whose sizes are below the recursive circuit size threshold implemented within Sonobe.

We can see that the user circuits with both hash functions are much smaller than the aggregator circuit and close to the recursion overhead. This allows us

to significantly boost the performance of user circuits by increasing the batch size. Indeed, with SHA256, the per transaction prover time is reduced from 4.1 seconds for batch size 1 to 0.88 seconds for batch size 10, which is a $4.6\times$ improvement. The RAM usage is also highly practical, with a maximum of 1050 MB for batch size 10. All these results confirm that PlasmaFold enables efficient client-side proving, which is crucial for real-world deployment.

Throughput. Regarding throughput, recall that a block in PlasmaFold is defined as $blk_t = (t, r_t^{\text{utxo}}, r_t^{\text{tx}}, r_t^{\text{signer}}, \mathcal{L}_t^{\text{signer}}, \mathcal{L}_t^{\text{deposit}}, \mathcal{L}_t^{\text{withdraw}})$. We consider the case where blk_t is only composed of transactions (no deposits or withdrawals) and all senders have signed r_t^{tx} . Here, t is obtained on the rollup contract L1 and $r_t^{\text{utxo}}, r_t^{\text{tx}}, r_t^{\text{signer}}$ are each 32 bytes values. The aggregator posts the content of $\mathcal{L}_t^{\text{signer}}$ for the contract to compute σ_t^{signer} . For our throughput estimation, we re-use the Intmax’s [27] proposed user-id size of ~ 4.15 -byte integer representing the user id , which makes it possible for the L2 to accomodate for more than the current earth population. Assuming that we use MicroNova [32] for the decider SNARK, the compressed proof size hovers at 12 KB given the aggregator circuit size. To compute the throughput of our L2, we divide the amount of data which can be inserted into an L1 block by the number of seconds occurring between two blocks [28]. In the case of Ethereum, within a blob of size 0.375 MB and a block time of 12 s, we have:

$$\frac{0.375 \text{ MB} - 12000 \text{ B} - 96 \text{ B}}{4.15 \text{ B} \times 12 \text{ s}} \approx 7300 \text{ TPS}$$

Moreover, Ethereum’s recent Pectra upgrade, which includes EIP-7691, doubled the average amount of available data, placing PlasmaFold at around 14000 TPS. By re-using Intmax’s short-id trick we are able to still reach high theoretical throughput, while featuring the above described efficient server and client proving routines, running on commodity hardware.

Instant synchronization. In contrast to previous work [27], since PlasmaFold features an aggregator which verifiably builds block and client-side proving routines working in the IVC model, any user can instantly synchronize to the chain tip. In Intmax, users need to run synchronization proofs, attesting to the user’s absence of action within each block. On an m2 pro such synchronization proof required a 4s proving time³: a user skipping blocks for a single day will need to run an 8h proving job. On the contrary, PlasmaFold setup does not require any synchronization proof: users are only required to prove deposit, send or withdraw transactions, without processing any blocks empty of any of their own actions.

5 Related Work

We discuss below existing constructions of L2s. These solutions aim to scale the L1 blockchain by 1) offloading execution from the L1, thereby relieving network nodes from the burden of re-executing every computation, and 2) minimizing

³<https://github.com/InternetMaximalism/intmax2-zkp>, main branch, 8af18ec

the amount of data posted on the L1 per transaction, allowing more transactions to fit within the L1’s fixed block size.

Existing L2 designs can be classified along two primary dimensions [23]: where they store transaction data (on-chain vs. off-chain) and how they attest to the validity of off-chain computations (fraud proofs vs. validity proofs).

The former dimension determines the L2 *data availability* [10, 11, 28] regime. An L2 with higher data availability costs tends to post more data on-chain, resulting in lower throughput, as throughput is fundamentally limited by the L1’s fixed block size. Conversely, while keeping most data off-chain can achieve higher throughput, it increases the risk of data unavailability, potentially leading to users losing access to their funds.

The *trust model* of an L2 depends on the latter dimension. L2s based on validity proofs operate on a “distrustful” model, where state transitions are considered invalid by default. The L2 operator must generate and post a cryptographic proof to the L1 to demonstrate that all transactions are valid and have been processed honestly. In contrast, L2s using fraud proofs adopt an “innocent until proven guilty” model. Participants can post new state updates without an upfront proof of validity. Instead, a challenge window is introduced, during which any observer can submit a fraud proof to demonstrate the invalidity of a state update and penalize the malicious actor.

Optimistic rollups. Optimistic rollups [17, 24, 31] are a class of L2 solutions that use fraud proofs to secure transactions. A centralized operator, often called a sequencer, is responsible for ordering and executing transactions. It periodically posts batches of transactions and the resulting state to the L1. Following this, the operator opens a time-delayed challenge window, allowing any honest participant to submit a fraud proof to the L1 if any transactions are invalid. The main drawback of optimistic rollups is their large on-chain data footprint. Although some optimistic rollups (e.g., Optimism [25]) compress transaction batch data before posting it to the L1 while still ensuring users can reconstruct and monitor the current rollup state, their throughput remains suboptimal (e.g., a theoretical maximum of ~ 700 TPS for Optimism).

Plasma. Plasma [13, 26] is an earlier framework for off-chain scaling that, similar to optimistic rollups, relies on fraud proofs. In Plasma, the L2 operator posts only a commitment to the L2 state to the L1, while coordinating with other users to ensure transaction data has been made available to them, without broadcasting it on the L1. However, users are consequently required to individually store relevant transaction data and also need to actively monitor the operator to challenge any malicious behavior. If the operator misbehaves or withholds data, users must engage in a complex exit game to withdraw their funds.

Zero-knowledge rollups. Another line of work [4, 10, 19, 22] utilizes SNARKs to prove the integrity of state transitions. This type of L2 is colloquially known as a *zero-knowledge rollup* or *validity rollup*. While offering stronger security guarantees than optimistic rollups, zero-knowledge rollups incur significant costs for proof generation. Furthermore, as they are designed for high data

availability [33], their on-chain data footprint is large, even with transaction data compression [34].

Among zero-knowledge rollups, several constructions are based on IVC [8, 9, 18]. The IVC framework is well-suited for blockchains, as it supports efficient and incremental proof generation that aligns with the sequential nature of block-based state updates. For instance, [9] proposes the concept of incrementally verifiable ledger systems (IVLS), which captures the efficiency of IVC but is tailored for blockchains. It notably features a compiler that utilizes IVC to transform a given ledger system (with many participating nodes) into an IVLS scheme. Another example, [8], leverages IVC and introduces the notion of “incrementally-computable SNARKs” for a succinct blockchain, reducing the cost of verifying the entire history of the state transition function from linear to constant time.

Furthermore, Intmax2 [27], a recent zero-knowledge rollup, attempts to combine the low on-chain data footprint of Plasma with the security and instant exit features of zero-knowledge rollups using Proof-Carrying Data (PCD). It notably introduced (1) a short-id trick minimizing onchain data requirements and (2) a data availability procedure guaranteeing the aggregator to process transactions where users acknowledged reciprocity of their transaction data and which we leveraged to obtain PlasmaFold offline guarantees and scalability. However, as seen above, the Intmax approach requires users to run a heavy PCD prover on their devices, imposing significant hardware requirements. In addition to this, users need to run impractical synchronization proofs, further increasing the computational burden on users’ devices and degrading the overall usability of the protocol.

Validium. A Validium [14] operates similarly to a zero-knowledge rollup and uses validity proofs as well to guarantee computational integrity, ensuring that the operator cannot steal funds or execute invalid state transitions.

The key difference is that in a Validium, the L2 operator only posts an updated chain tip on L1 attesting to the honest execution of the state transition function, while the underlying transaction data is kept off-chain and managed by several storage nodes. To provide data availability, Validium either requires users to trust these nodes or relies on complex game-theoretic and cryptoeconomic mechanisms. Nevertheless, users still face the risk of their funds being frozen if storage nodes decide to withhold data.

6 Conclusion

In this paper, we presented PlasmaFold, a novel L2 solution that achieves low on-chain data footprint and high throughput while ensuring data availability and featuring instant, non-interactive exit game. These properties are made possible thanks to our hybrid architecture that leverages both server-side and client-side proving with IVC. The evaluation on our implementation confirms the practicality of PlasmaFold, demonstrating that it can achieve a throughput of up to 14000 TPS on Ethereum, while the client-side balance proof only requires less than 1 s to generate in a browser.

References

- [1] 0xPARC and Privacy & Scaling Explorations. **Sonobe**: an experimental folding schemes library, 2023.
- [2] arkworks contributors. **arkworks** zksnark ecosystem, 2022.
- [3] Arasu Arun and Srinath Setty. Nebula: Efficient read-write memory and switchboard circuits for folding schemes. Cryptology ePrint Archive, Paper 2024/1605, 2024.
- [4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.
- [5] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Scalable zero knowledge via cycles of elliptic curves. *Algorithmica*, 79:1102–1160, 2017.
- [6] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for snarks and proof-carrying data. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, pages 111–120, 2013.
- [7] Dan Boneh and Binyi Chen. LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. Cryptology ePrint Archive, Paper 2024/257, 2024.
- [8] Joseph Bonneau, Izaak Meckler, Vanishree Rao, and Evan Shapiro. Coda: Decentralized cryptocurrency at scale. Cryptology ePrint Archive, Paper 2020/352, 2020.
- [9] Weikeng Chen, Alessandro Chiesa, Emma Dauterman, and Nicholas P. Ward. Reducing participation costs via incremental verification for ledger systems. Cryptology ePrint Archive, Paper 2020/1522, 2020.
- [10] Kyle Croman, Christian Decker, Ittay Eyal, Adem Efe Gencer, Ari Juels, Ahmed Kosba, Andrew Miller, Prateek Saxena, Elaine Shi, Emin Gün Sirer, et al. On scaling decentralized blockchains: (a position paper). In *International conference on financial cryptography and data security*, pages 106–125. Springer, 2016.
- [11] Christian Decker and Roger Wattenhofer. Information propagation in the bitcoin network. In *IEEE P2P 2013 Proceedings*, pages 1–10. IEEE, 2013.
- [12] Sergi Delgado-Segura, Cristina Pérez-Solà, Guillermo Navarro-Arribas, and Jordi Herrera-Joancomartí. Analysis of the bitcoin UTXO set. Cryptology ePrint Archive, Paper 2017/1095, 2017.
- [13] Ethereum Foundation. Plasma. <https://ethereum.org/en/developers/docs/scaling/plasma/>.
- [14] Ethereum Foundation. Validium. <https://ethereum.org/en/developers/docs/scaling/validium/>.

- [15] Alex Gluchowski. zkrollup vs. validium. *Online*. Disponible en: <https://medium.com/matter-labs/zkrollup-vs-validium-starkey-5614e38bc263>, 2020.
- [16] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for {Zero-Knowledge} proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535, 2021.
- [17] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S Matthew Weinberg, and Edward W Felten. Arbitrum: Scalable, private smart contracts. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1353–1370, 2018.
- [18] Assimakis Kattis and Joseph Bonneau. Proof of necessary work: succinct state verification with fairness guarantees. In *International Conference on Financial Cryptography and Data Security*, pages 18–35. Springer, 2023.
- [19] Ahmed Kosba, Zhichao Zhao, Andrew Miller, Yi Qian, Hubert Chan, Charalampos Papamanthou, Rafael Pass, Elaine Shi, et al. CØcØ: A framework for building composable zero-knowledge proofs. *Cryptology ePrint Archive*, 2015.
- [20] Abhiram Kothapalli and Srinath Setty. Hypernova: Recursive arguments for customizable constraint systems. In *Annual International Cryptology Conference*, pages 345–379. Springer, 2024.
- [21] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.
- [22] Patrick McCorry, Chris Buckland, Bennet Yee, and Dawn Song. SoK: Validating bridges as a scaling solution for blockchains. *Cryptology ePrint Archive*, Paper 2021/1589, 2021.
- [23] Kamilla Nazirkhanova, Joachim Neu, and David Tse. Information dispersal with provable retrievability for rollups. In *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*, pages 180–197, 2022.
- [24] Optimism. Chain architecture. "<https://docs.optimism.io/operators/chain-operators/architecture>".
- [25] Optimism. Optimism specs - execution engine. <https://specs.optimism.io/protocol/fjord/exec-engine.html>.
- [26] Joseph Poon and Vitalik Buterin. Plasma: Scalable autonomous smart contracts. *White paper*, pages 1–47, 2017.
- [27] Erik Rybakken, Leona Hioki, Mario Yaksetig, Denisa Diaconescu, František Silváši, and Julian Sutherland. Intmax2: A ZK-rollup with minimal onchain data and computation costs featuring decentralized aggregators. *Cryptology ePrint Archive*, Paper 2023/1082, 2023.
- [28] Cosimo Sguanci, Roberto Spatafora, and Andrea Mario Vergani. Layer 2 blockchain scaling: a survey, 2021.

- [29] Louis Tremblay Thibault, Tom Sarry, and Abdelhakim Senhaji Hafid. Blockchain scaling using rollups: A comprehensive survey. *IEEE Access*, 10:93039–93054, 2022.
- [30] Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008. Proceedings 5*, pages 1–18. Springer, 2008.
- [31] Zhe Ye, Ujval Misra, Jiajun Cheng, Wenyang Zhou, and Dawn Song. Specular: Towards secure, trust-minimized optimistic blockchain execution, 2024.
- [32] Jiaxing Zhao, Srinath Setty, Weidong Cui, and Greg Zaverucha. Micronova: Folding-based arguments with efficient (on-chain) verification. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1964–1982. IEEE, 2025.
- [33] zkSync. zksync - data availability. <https://docs.zksync.io/zksync-protocol/rollup/data-availability>.
- [34] zkSync. zksync - pubdata compression. <https://docs.zksync.io/zksync-protocol/rollup/pubdata-compression>.